Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №4
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Бинарное дерево поиска»

Выполнили:
студенты группы 25ВВВ2
Сабыров Т.Т.
Малышев Н.Д.

Приняли:
к.т.н., доцент Юрова О.В.
ст. преподаватель Деев М.В.

Пенза 2026

1. Цель работы

Изучение принципов организации и функционирования динамических нелинейных структур данных на примере бинарного дерева поиска (Binary Search Tree). Освоение рекурсивных методов создания дерева, визуализации структуры, поиска элементов по значению, подсчета частоты вхождений, исключения дубликатов, а также анализ теоретической сложности операций поиска.

2. Лабораторное задание

1. Разработать интерактивное меню программы для выбора режима построения дерева: 1 — дерево с дубликатами, 2 — дерево без дубликатов (дополнительное задание).

2. Реализовать автоматическую генерацию заданного пользователем количества элементов дерева случайными числами.

3. Реализовать функцию визуализации дерева на экране, повернутого на 90 градусов (корень слева).

4. Организовать меню операций над деревом: 1 — поиск элемента по значению с определением глубины и количества вхождений, 2 — добавление одного элемента (с проверкой на уникальность в режиме без дубликатов), 3 — возврат к меню выбора дерева.

5. Дополнительное задание 1*: изменить функцию добавления элементов для исключения добавления одинаковых значений (дубликатов).

6. Дополнительное задание 2*: провести детальный теоретический анализ временной и пространственной сложности процедуры поиска по значению в бинарном дереве.

3. Ход работы

3.1. Описание используемых алгоритмов и структур данных

Бинарное дерево поиска (BST) представляет собой связанную динамическую структуру данных, состоящую из узлов (Node), каждый из которых содержит информационное поле (ключ data) и два указателя: left (на левое поддерево) и right (на правое поддерево). Для любого узла выполняется фундаментальное свойство порядка: ключи всех узлов левого поддерева строго меньше ключа родителя, а ключи всех узлов правого поддерева больше либо равны ключу родителя.

Алгоритм добавления узла CreateTree() является рекурсивным: если текущий указатель равен NULL, создается новый узел через malloc(), его полям присваиваются начальные значения, и он возвращается в качестве корня поддерева. Если значение ключа меньше ключа в узле, управление передается рекурсивно левому потомку, иначе — правому.

Алгоритм предотвращения дубликатов CreateTreeUnique() проверяет равенство вставляемого значения текущему узлу. Если значения совпадают, вставка не происходит, выставляется флаг added = 0, что исключает избыточное дублирование данных в дереве.

Алгоритм поиска search_tree() использует упорядоченность дерева: на каждом шаге искомый ключ сравнивается со значением текущего узла. При совпадении узел считается

найденным. Если искомое значение меньше, поиск продолжается только в левом поддереве, если больше — только в правом. Это сокращает область поиска вдвое на каждом шаге в сбалансированном дереве.

Детальный теоретический анализ сложности процедуры поиска (Задание 2*):

1. Временная сложность поиска напрямую определяется высотой дерева h, поскольку при каждом рекурсивном переходе выполняется константное число элементарных операций O(1), а максимальная длина пути от корня до листа равна h. Таким образом, временная сложность поиска составляет T(N) = O(h).

2. Средний случай (сбалансированное дерево): если элементы добавляются в случайном порядке, дерево получается приближенно сбалансированным. В идеально сбалансированном дереве на каждом уровне k число узлов равно 2^k, а суммарное число узлов N = 2^(h+1) - 1. Отсюда высота дерева h = floor(log2(N)). Математическое ожидание длины пути поиска для случайных деревьев составляет E[h] ≈ 1.39 * log2(N). Следовательно, в среднем случае временная сложность поиска составляет O(log N).

3. Наихудший случай (вырожденное дерево): если элементы поступают в монотонно возрастающем или убывающем порядке (например, 1, 2, 3, ...), то каждый новый элемент становится исключительно правым (или левым) потомком. В результате дерево вырождается в связный список высотой h = N. В этом случае поиск требует последовательного обхода всех N узлов, и временная сложность деградирует до линейной O(N).

4. Наилучший случай: искомый элемент оказывается расположенным непосредственно в корневом узле дерева. В этом случае поиск завершается за один шаг, сложность составляет O(1).

5. Пространственная сложность (память стека вызовов): поскольку поиск реализован рекурсивно, в стеке вызовов сохраняется цепочка фреймов глубиной, равной глубине узла. Пространственная сложность составляет O(h) дополнительной памяти, что соответствует O(log N) для сбалансированного дерева и O(N) в наихудшем случае вырождения.

3.2. Оценка сложности процедуры поиска по значению в бинарном дереве

4. Оценить сложность процедуры поиска по значению в бинарном дереве.

Случай	**Высота h**	**Сложность**
Лучший (значение в корне)	—	O(1)
Средний (сбалансированное дерево)	$\log_2 n$	O(log n)
Худший (вырожденное дерево, ввод по порядку)	n	O(n)

В среднем поиск работает за O(log n), но без балансировки в худшем случае (например, при вводе чисел по возрастанию) деградирует до O(n).

3.3. Описание используемых функций и структуры программы

struct Node: определение структуры узла бинарного дерева поиска (поля: data, left, right).

CreateTree(struct Node* root, int data): рекурсивная функция создания дерева и вставки нового элемента с поддержкой дубликатов.

CreateTreeUnique(struct Node* root, int data, int* added): модифицированная функция добавления элементов, исключающая дубликаты и сигнализирующая о статусе добавления через указатель added.

print_tree_helper(struct Node* r, char* prefix, int is_left) и print_tree(struct Node* r): наглядный графический вывод дерева на консоль с четкими ветвями (/-- и \--) и вертикальными линиями связности, выдерживающими постоянный шаг между родителями и потомками.

search_tree(struct Node* root, int key, int* depth): функция поиска ключа в дереве с фиксацией уровня глубины узла.

count_occurrences(struct Node* root, int key): функция рекурсивного подсчета количества вхождений заданного ключа в дереве.

free_tree(struct Node* root): функция рекурсивного освобождения динамической памяти всех узлов дерева.

main(void): точка входа программы, реализующая интерактивное консольное меню с выбором типа дерева, автоматической генерацией элементов, поиском, добавлением и возвратом в главное меню.

4. Результаты работы программы

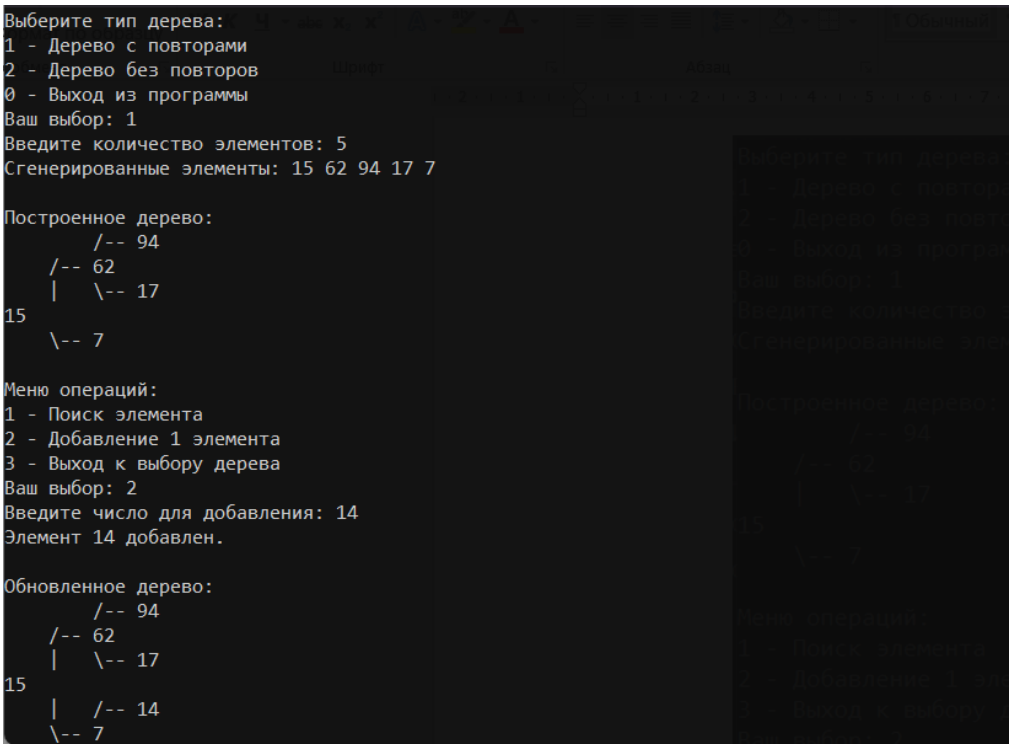


Рисунок 1 – Результат работы программы 1

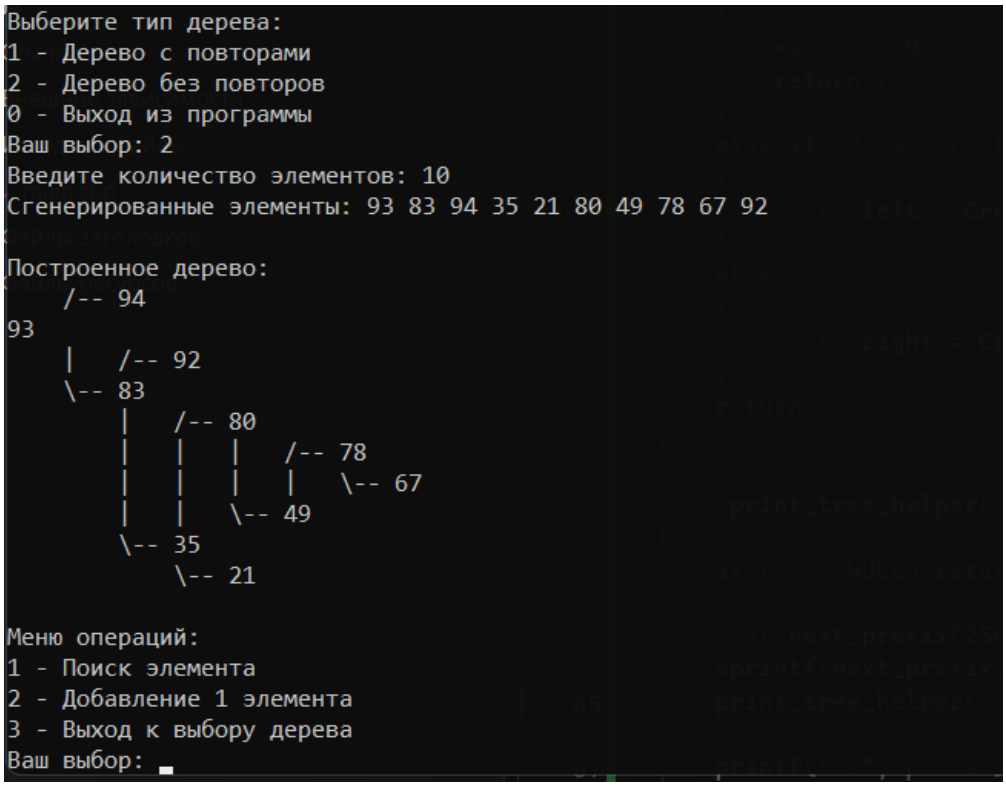


Рисунок 2 – Результат работы программы 2

5. Выводы

В ходе выполнения лабораторной работы закреплены навыки работы с динамическими рекурсивными структурами данных на языке Си/C++.

Разработана интерактивная программа с удобным двухуровневым консольным меню, автоматической генерацией ключей случайными числами, наглядной визуализацией дерева, поиском узлов и динамическим добавлением элементов.

Реализована и апробирована функция CreateTreeUnique(), предотвращающая дублирование ключей в дереве поиска.

Выполнен детальный теоретический анализ сложности процедуры поиска по значению в BST: показано, что для сбалансированного дерева сложность составляет O(log N), при вырождении в список — O(N), а в лучшем случае — O(1).